

2. Les notions mentales de base

2.1. Objet réel et objet informatique

En POO, on s'inspire du monde réel pour organiser le code.

1. Objet réel

Un **objet réel** est une chose que l'on peut observer dans la vie courante.

Exemples :

- une voiture
- un téléphone
- un étudiant
- un compte bancaire

Un objet réel possède généralement :

- des **caractéristiques**
- des **actions**

Exemple : une voiture

- Caractéristiques : couleur, marque, vitesse
- Actions : démarrer, accélérer, freiner

2. Objet informatique

Un **objet informatique** est une représentation d'un objet réel dans un programme.

Il contient :

- des **données** → appelées **attributs**

- des **actions** → appelées **méthodes**

Exemple :

Objet : Voiture

Attributs :

- couleur
- marque
- vitesse

Méthodes :

- demarrer()
- accélérer()
- freiner()

3. Idée à retenir

Un objet informatique est une **version simplifiée d'un objet réel**, créée pour résoudre un problème dans un programme.

On ne garde que ce qui est utile.

Exemple :

Pour une application de location de voiture, on peut garder :

marque

modèle

prix

disponibilité

Mais on ignore peut-être :

couleur des sièges

type de parfum dans la voiture

Formule simple à mémoriser

Objet réel = chose du monde réel

Objet informatique = représentation utile de cette chose dans le programme

2.2. État d'un objet

L'état d'un objet correspond aux **valeurs actuelles de ses attributs**.

Autrement dit :

État = ce que l'objet "a" à un moment donné

Exemple simple

Objet : **Voiture**

Attributs :

couleur = "rouge"

vitesse = 0

moteurAllumé = false

À ce moment précis, l'état de la voiture est :

rouge, arrêtée, moteur éteint

L'état peut changer

Quand on utilise l'objet, ses valeurs peuvent évoluer.

Exemple :

```
moteurAllumé = true
```

```
vitesse = 50
```

Le nouvel état devient :

```
rouge, en mouvement, moteur allumé
```

Idée à retenir

Un objet a des **attributs**.

Les **valeurs de ces attributs** forment son **état**.

Formule simple :

Attributs = ce que l'objet possède

État = valeurs actuelles des attributs

2.3. Comportement d'un objet

Le **comportement d'un objet** correspond à ce que l'objet **peut faire**.

En programmation, ce comportement est représenté par des **méthodes**.

Comportement = actions possibles de l'objet

Exemple simple

Objet : **Voiture**

Comportements possibles :

```
demarrer()
```

```
accellerer()
```

```
freiner()
```

`arreter()`

Ces méthodes permettent à la voiture d'agir ou de changer son état.

Exemple :

`accellerer()`

peut modifier :

`vitesse = 0`

en :

`vitesse = 50`

Idée à retenir

Un objet possède :

des attributs → son état

des méthodes → son comportement

Formule simple :

État = ce que l'objet a

Comportement = ce que l'objet fait

2.4. Identité d'un objet

L'**identité d'un objet** signifie que chaque objet est **unique**, même s'il ressemble à un autre.

En POO, deux objets peuvent avoir le **même état**, mais rester **deux objets différents**.

Identité = ce qui permet de reconnaître un objet comme unique

Exemple simple

On crée deux voitures :

```
voiture1 :
```

```
couleur = "rouge"
```

```
marque = "Toyota"
```

```
voiture2 :
```

```
couleur = "rouge"
```

```
marque = "Toyota"
```

Elles ont les mêmes valeurs, donc le même état.

Mais ce sont deux voitures différentes :

```
voiture1 ≠ voiture2
```

Comme deux téléphones identiques en magasin : même modèle, même couleur, mais pas le même objet.

Idée à retenir

Un objet possède :

un état → ses valeurs actuelles

un comportement → ses actions

une identité → son existence unique

Formule simple :

Même état ne veut pas dire même objet.

2.5. Responsabilité d'un objet

La **responsabilité d'un objet** correspond à ce que l'objet est **chargé de savoir** ou **de faire** dans le programme.

Responsabilité = rôle de l'objet dans le programme

Exemple simple

Objet : **CompteBancaire**

Il peut être responsable de :

connaître le solde

déposer de l'argent

retirer de l'argent

vérifier si le solde est suffisant

Donc, ce n'est pas à un autre objet de modifier directement son solde n'importe comment.

Pourquoi c'est important ?

Une bonne responsabilité rend le code :

plus clair

plus organisé

plus facile à modifier

Chaque objet doit avoir un rôle précis.

Idée à retenir

Un objet ne doit pas tout faire.

Il doit faire ce qui correspond à son rôle.

Formule simple :

Un bon objet = un rôle clair + des responsabilités cohérentes

2.6. Message envoyé à un objet

En POO, un **message envoyé à un objet** signifie qu'on demande à l'objet de **faire quelque chose**.

En pratique, cela correspond souvent à l'appel d'une **méthode**.

Message = demande envoyée à un objet

Exemple simple

Objet : **Voiture**

On lui envoie le message :

`demarrer()`

Cela veut dire :

"Voiture, démarre."

Autres exemples :

`accélérer()`

`freiner()`

`arrêter()`

Chaque message demande à l'objet d'exécuter un comportement.

Idée à retenir

Un objet reçoit un **message**.

Il répond en exécutant une **méthode**.

Formule simple :

Envoyer un message = appeler une méthode sur un objet

2.7. Collaboration entre objets

La **collaboration entre objets** signifie que plusieurs objets travaillent ensemble pour réaliser une action.

Un objet ne fait pas toujours tout seul : il peut demander de l'aide à d'autres objets.

Collaboration = objets qui s'envoient des messages pour travailler ensemble

Exemple simple

Dans une application de commande :

Client

Commande

Produit

Paielement

Ces objets collaborent :

Le Client crée une Commande.

La Commande contient des Produits.

Le Paiement valide la Commande.

Chaque objet garde son rôle.

Pourquoi c'est important ?

La collaboration permet de créer un programme :

plus organisé

plus réaliste

plus facile à comprendre

Au lieu d'avoir un seul objet qui fait tout, on répartit le travail.

Idée à retenir

Un bon programme orienté objet ressemble à une équipe :

chaque objet a un rôle

chaque objet peut communiquer

chaque objet participe au résultat final

Formule simple :

Collaborer = se parler pour agir ensemble

2.8. Cycle de vie d'un objet

Le **cycle de vie d'un objet** décrit les grandes étapes de son existence dans un programme.

Cycle de vie = naissance, utilisation, disparition d'un objet

1. Naissance de l'objet

Un objet naît quand on le **crée**.

Exemple :

```
voiture = nouvelle Voiture()
```

À ce moment, l'objet existe en mémoire.

2. Utilisation de l'objet

On utilise l'objet en lisant ses attributs ou en appelant ses méthodes.

Exemple :

```
voiture.demarrer()
```

```
voiture.accelerer()
```

L'objet peut alors changer d'état.

3. Disparition de l'objet

Un objet disparaît quand il n'est plus utile ou plus accessible.

Dans beaucoup de langages modernes, la mémoire est libérée automatiquement.

Idée à retenir

Un objet a une “vie” dans le programme :

création → utilisation → disparition

Formule simple :

Un objet naît, agit, puis disparaît.

2.9. Modéliser un problème avec des objets

Modéliser un problème avec des objets, c'est transformer une situation réelle en objets informatiques.

Modéliser = représenter un problème sous forme d'objets

Exemple simple

Problème : gérer une bibliothèque.

On peut trouver les objets importants :

Livre

Lecteur

Emprunt

Bibliothécaire

Chaque objet a un rôle :

Livre → contient un titre, un auteur

Lecteur → emprunte des livres

Emprunt → garde la date d'emprunt

Comment faire ?

Il faut se poser 3 questions simples :

Quels sont les objets importants ?

Que savent-ils ?

Que peuvent-ils faire ?

Exemple :

Objet : Livre

Il sait : titre, auteur, disponibilité

Il peut : être emprunté, être retourné

Idée à retenir

Modéliser, c'est découper un problème en objets qui ont chacun :
un état

un comportement

une responsabilité

Formule simple :

Un problème réel → des objets → du code mieux organisé

2.10. Penser en responsabilités plutôt qu'en instructions

En POO, on ne pense pas seulement :

Étape 1 → faire ceci

Étape 2 → faire cela

Étape 3 → faire autre chose

On pense plutôt :

Quel objet est responsable de faire quoi ?

Exemple simple

Problème : valider une commande.

Au lieu de penser :

1. vérifier les produits
2. calculer le total
3. faire le paiement
4. confirmer la commande

On répartit les responsabilités :

Commande → calcule le total

Produit → connaît son prix

Paieement → valide le règlement

Client → passe la commande

Pourquoi c'est important ?

Cela rend le programme :

plus organisé

plus logique

plus facile à modifier

Chaque objet a un rôle clair.

Idée à retenir

En POO, on évite de tout mettre au même endroit.

On distribue le travail aux bons objets.

Formule simple :

Ne demande pas seulement : "Quelles étapes ?"

Demande plutôt : "Quel objet est responsable ?"